Curiosity-Driven Autonomous Game Agent for Pokémon FireRed: A Biologically-Inspired Architecture with Interpretable Perceptrons, Hebbian Learning, and Markov Demonstration Matching

Angshuman Basu
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
basu.a@northeastern.edu

Nathaniel Maw
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
maw.n@northeastern.edu

Achal Mukkapati
Khoury College of Computer Sciences
Northeastern University
Boston, MA, USA
mukkapati.a@northeastern.edu

***Abstract*—Autonomous agents capable of navigating full-length role-playing video games (RPGs) represent a significant open challenge in artificial intelligence, demanding long-horizon planning, context switching across heterogeneous subsystems, and adaptation to partial observability—all without a dense reward signal. We present a curiosity-driven autonomous agent that plays Pokémon FireRed for the Game Boy Advance via the BizHawk emulator, driven entirely by intrinsic motivation and structured human demonstration matching, with no externally defined reward function, reward shaping, or score-based optimization. The agent is built around a biologically-inspired philosophy: exploration emerges from novelty and curiosity; learning proceeds through Hebbian co-activation reinforcement; decisions remain fully interpretable via single-layer perceptrons with empirically discovered activation functions; and behavioral scaffolding is provided by Markov chains extracted from human gameplay demonstrations, scoped to current game progress via an Adaptive Checkpoint Window System. The system operates through a Lua–Python pipeline interfacing with BizHawk, reading structured game state from GBA memory addresses and issuing controller inputs in real time. In an 8,255-step evaluation run, the agent autonomously navigated eleven game maps from Pallet Town through Viridian Forest, achieved a 77.8% battle win rate over 18 encounters, maintained a deduplicated stagnation ratio of 71.4% compared to a human baseline of 78.7%, and arrived at the majority of map checkpoints with full party HP. Key contributions include a fully curiosity-driven agent for a complete RPG with no reward shaping; an interpretable multi-pool perceptron pipeline with context-specific modules for overworld, battle, bag, and party subsystems; empirical activation function discovery with parametric warping, a novel approach requiring no gradient computation through activation parameters; Hebbian learning with dual-timescale gradient updating in the battle pipeline; and an Adaptive Checkpoint Window System that scopes demonstration matching to the agent's current game stage.**

***Index Terms*—curiosity-driven exploration, Hebbian learning, intrinsic motivation, game-playing AI, interpretable machine learning, imitation learning, Markov matching**

I. Introduction

The aspiration to build artificial agents capable of playing video games has long served as a productive proxy for broader questions in AI research. Games provide controlled, reproducible environments with rich state spaces and meaningful decision-making demands. Yet the overwhelming majority of progress in game-playing AI has been concentrated in a narrow class of environments: Atari 2600 games with dense pixel rewards [1], board games with perfect information and well-defined terminal states [2], and real-time strategy games with measurable win conditions [3]. These settings share structural properties that make them tractable for standard reinforcement learning (RL): bounded state spaces, frequent rewards, and quantifiable scalar performance.

Narrative-driven RPGs occupy an entirely different region of that space. Pokémon FireRed [4] presents an agent with partial observability across more than twenty interconnected areas; context switching among qualitatively different interaction modes including overworld navigation, turn-based combat, menu-driven inventory management, and NPC dialogue; time horizons measured in hours before any milestone is reached; and narrative-gated progression where specific event sequences must be completed before new world regions become accessible. Crucially, there is no scalar reward to optimize. Standard RL approaches are poorly suited to this setting: sparse rewards fail to guide learning over long horizons, reward shaping introduces strong human-designed assumptions about optimal behavior, and opaque deep networks resist interpretation. To our knowledge, no prior published work has attempted to play a full-length Pokémon game with a reward-free curiosity-driven approach.

This paper presents an alternative paradigm grounded in a biological metaphor: the agent learns to navigate its environment the way a living organism does—through curiosity,

habituation, pattern recognition, and imitation—rather than through numerical reward optimization. This philosophy unifies every architectural decision in the system. The agent explores Pokémon FireRed without a reward function, driven by novelty-seeking and intrinsic curiosity. It learns behavioral patterns by imitating human demonstrations represented as Markov transition chains. Its decision-making components are single-layer perceptrons chosen for full interpretability, updated via Hebbian learning rules inspired by the canonical biological principle that "neurons that fire together, wire together" [11]. Its activation functions are not assumed but discovered empirically through gameplay experience.

II. Related Work

The system described in this paper sits at the intersection of three bodies of literature: intrinsic motivation and curiosity-driven exploration, imitation learning from human demonstration, and lightweight online learning with adaptive architectures. Rather than surveying these areas broadly, we focus on the specific works whose methods and insights are directly adopted or deliberately departed from in our design.

A. Intrinsic Motivation and Curiosity-Driven Exploration

The foundational challenge motivating our system is the sparse reward problem in reinforcement learning: when extrinsic feedback from the environment is rare or absent, agents that rely purely on environmental signals fail to explore productively. Oudeyer and Kaplan [5] provide a formal taxonomy of intrinsic motivation mechanisms, distinguishing between novelty-based, surprise-based, and learning-progress-based exploration drives. Their key finding is that internal signals—generated from the agent's own predictions about the world rather than from environmental feedback—can sustain directed exploration across extended time horizons. This is the theoretical bedrock of our system: our agent operates with no externally defined reward, relying instead on state-change magnitude as a proxy for environmental novelty. We adopt Oudeyer and Kaplan's core principle that curiosity should emerge from the agent's own model of its world, but implement it through a lightweight symbolic signal (weighted L2 norm over verified GBA memory-read state vectors) rather than the continuous learned models their framework envisions.

Pathak et al. [6] operationalized curiosity as a practical RL objective through the Intrinsic Curiosity Module (ICM). ICM pairs a forward model—predicting the next latent state given the current state and action—with an inverse model that infers the action that caused a given state transition. The inverse model's critical role is to learn a feature space invariant to aspects of the environment the agent cannot control (visual noise, background dynamics), preventing the "noisy TV" problem where uncontrollable stochasticity inflates the curiosity signal. In Pokémon FireRed, we face an analogous problem: palette flickers and tile animations produce large changes in the raw visual state without conveying meaningful spatial or semantic information. Rather than learning an invariant feature space as ICM does, we address this by maintaining two separate novelty channels—spatial novelty from tile visit counts over memory coordinates, and contextual novelty from state-change magnitude over the semantic feature vector—and weighting them independently in the curiosity score. This is a deliberate simplification that foregoes ICM's representational generality in exchange for full interpretability and compatibility with real-time emulator play on consumer hardware without GPU acceleration.

Burda et al. [8] demonstrated through a large-scale empirical study that pure curiosity-driven learning—with no extrinsic reward at all—can achieve surprisingly strong performance across a wide range of environments, but also identified systematic failure modes. Most critically, they showed that curiosity alone collapses in environments with strong narrative structure or procedural gating, where the sequence of reachable states is tightly constrained by story events. This finding directly shaped the architecture of our system: rather than relying on curiosity alone, we use human demonstration matching as the primary scaffolding mechanism, with curiosity serving as the fallback for states not covered by demonstrations. Our empirical observation during development—that pure curiosity-driven exploration failed to navigate narrative-gated Pokémon FireRed—reproduces Burda et al.'s finding in a new domain.

Bellemare et al. [7] provided a theoretical unification of count-based exploration and intrinsic motivation, showing that exploration bonuses inversely proportional to state visitation frequency can be derived from pseudo-count estimates over learned density models. Our per-tile visitation map implements the same qualitative mechanism at a symbolic level: tiles visited fewer times receive higher novelty scores and higher action priority. By maintaining explicit visit counts in memory rather than learning a density model, we trade generality for interpretability—the agent's novelty assessment of any tile can be read directly from its visit counter, with no density model to inspect or debug.

B. Imitation Learning and Learning from Demonstration

Hussein et al. [10] provide a comprehensive survey of imitation learning methods, organizing them along three axes: what information is available from the demonstrator, how the demonstration data is used, and what the output of learning is. Their survey identifies a consistent failure mode across imitation learning approaches: *distribution shift*, where the learned policy visits states not covered by the training demonstrations, causing compounding errors as the agent moves further from familiar territory. In a long-horizon game like Pokémon FireRed, where a single playthrough spans dozens of maps and hundreds of decision points, distribution shift is severe: demonstrations covering Pallet Town are structurally irrelevant when the agent is navigating Viridian Forest. Our Markov matching system directly addresses this challenge through the Adaptive Checkpoint Window System, which restricts matching to demonstrations whose game-progress context (badge count, team level, and spatial position) matches the agent's current state.

Ross et al. [9] formalized the distribution shift problem and provided the first algorithm with provable guarantees against it. Their Dataset Aggregation (DAgger) algorithm frames imitation learning as no-regret online learning: rather than training once on a fixed demonstration corpus and deploying the resulting policy, DAgger iteratively executes the current policy, queries the human expert on the states the policy actually visits, and appends this new expert data to the training set. The key DAgger insight we adopt is that the demonstration retrieval pool should reflect the states the agent currently visits—not the states a human happened to visit in a prior session. Rather than querying the human for live re-demonstration, we achieve the same effect offline through the Adaptive Checkpoint Window System, which filters the fixed demonstration corpus to only those frames whose progress context matches the agent's current state.

Miconi et al. [12] proposed differentiable plasticity, which integrates Hebbian learning directly into neural networks by augmenting each synapse with a plastic component trained end-to-end via backpropagation. We adopt Miconi et al.'s core insight—that Hebbian co-activation reinforcement enables fast adaptation that complements slower gradient-based consolidation—but diverge from their implementation in two ways. First, we do not use backpropagation to train the fixed weight component in the overworld, bag, and party pipelines; Hebbian updating is the *sole* weight update mechanism in those pipelines. Second, we implement Hebbian updating on single-layer perceptrons rather than on synapses within deep networks, ensuring that every learned association can be directly read from the weight vector.

C. Lightweight Online Learning and Adaptive Architectures

Oja [13] derived a normalized variant of the Hebbian learning rule with guaranteed convergence and stability properties absent in naive Hebb updates. The key contribution was showing that a single modification to the Hebbian update—subtracting a term proportional to the current weight magnitude weighted by the squared output—prevents weight explosion and causes the weight vector to converge to the first principal component of the input distribution. We incorporate Oja's stability principle through a clamped bias range $b \in [-2.0, 2.0]$ and an adaptive learning rate that decays when recent error is low and grows when error is high.

Izhikevich [14] addressed the distal reward problem in spiking neural network models using eligibility traces—transient chemical tags at recently co-activated synapses—as a bridge between immediate Hebbian plasticity and delayed reward signals. Our dual-timescale eligibility trace system ($\gamma_{\text{fast}} = 0.5$, $\gamma_{\text{slow}} = 0.95$) is a direct implementation of this mechanism in a non-spiking perceptron context. The fast trace enables within-battle tactical adaptation, while the slow trace consolidates type-effectiveness knowledge that transfers across opponents of a given type.

Borkar [15] provided the convergence theory for two-timescale stochastic approximation algorithms, showing that when two coupled iterative processes update at different rates, the faster process converges to a local equilibrium conditional on the slow process's current state, while the slower process updates accordingly. This two-timescale convergence result is the theoretical justification for our battle pipeline's split learning scheme: turn-level loss updates (fast timescale) govern immediate tactical decisions, while battle-level loss updates (slow timescale) consolidate strategic patterns about which opponents to fight and whether to flee.

D. Contributions

This paper makes the following primary contributions to the game-playing AI and intrinsic motivation literatures.

1) **Reward-free play in a full-length RPG.** We present, to our knowledge, the first published curiosity-driven agent that achieves meaningful progression in a complete narrative RPG with no reward function, reward shaping, or score-based objective of any kind.
2) **Empirical activation function discovery with parametric warping.** Each perceptron independently discovers its optimal activation function from a shared candidate library through correlation-based fitting against its own prediction error buffer, then refines it via greedy geometric warping of four parameters (scale, stretch, shift, offset). This technique requires no gradient computation through activation parameters and is distinct from both fixed-activation networks and end-to-end learned activation approaches.
3) **Interpretable multi-pool perceptron pipeline.** We introduce a hierarchical pool-and-pipeline architecture that organizes single-layer perceptrons into context-specific decision modules for overworld, battle, bag, and party subsystems, with authority-weighted fallback to demonstration matching and hardcoded defaults, ensuring every decision remains fully traceable to specific learned weights.
4) **Dual-timescale Hebbian and gradient learning.** The battle pipeline combines Hebbian co-activation reinforcement with a two-timescale backpropagation system operating at the turn level and battle level simultaneously, targeting disjoint pipeline pools and blended proportionally to pipeline authority [15].
5) **Adaptive Checkpoint Window System.** We introduce a progress-scoped demonstration matching mechanism that restricts the Markov retrieval pool to demonstrations whose game-progress context matches the agent's current state, directly resolving the cross-stage contamination failure mode identified in long-horizon imitation learning [9].
6) **Lua–Python emulator integration pipeline.** A complete GBA memory-read and action-dispatch pipeline interfacing with the BizHawk emulator via JSON file exchange and background threading, designed for portability to other GBA titles sharing similar memory layouts.

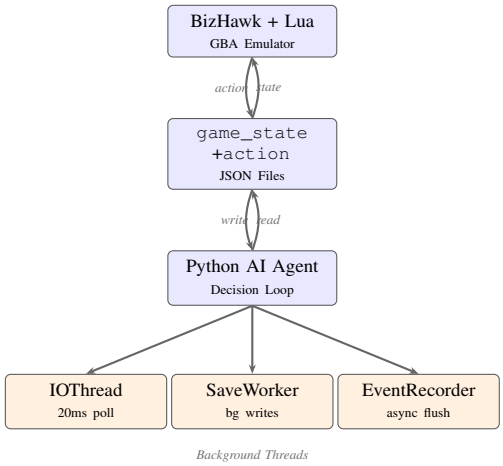

Fig. 1. System architecture. The Lua script writes game state to JSON and reads the agent's action response each frame. Three background threads decouple I/O, file saving, and event recording from the main decision loop.

III. Methods

A. System Architecture and Emulator Interface

The system establishes a bidirectional communication bridge between the BizHawk GBA emulator and a Python-based learning agent, as illustrated in Fig. 1. A Lua script embedded in the emulator reads verified GBA memory addresses every frame, extracting player coordinates from `0x02036E48`/`0x02036E4A`, the map identifier from `0x02036E44`, facing direction from `0x02036E50`, a battle flag from `0x0202000A`, and a game state register from `0x020204C2`. The script additionally reads GBA video memory: the background palette at `0x05000000` yields 256 RGB triples (768 values), and the BG0 tilemap at `0x06000000` yields a 30×20 tile identifier grid (600 values). Battle-specific data—cursor positions, species IDs, HP/PP values, move IDs, stat stages, and status conditions—are extracted from battle mirror addresses at `0x02023BE4` (player) and `0x02023C3C` (enemy). Party data comes from six 100-byte slots at `0x02024284`, and badge progress from the SaveBlock1 bitmask at offset `0x0FE4`.

All extracted data is serialized to `game_state.json` at approximately 30 frames per second. The Python agent reads this file, executes decision logic across the relevant pipeline modules, and writes `action.json`; the Lua script reads this on the following frame and calls BizHawk's `joypad.set()` with the corresponding button.

This architecture was chosen deliberately. By treating the Lua script as stable perception infrastructure and concentrating all learning and reasoning in Python, the system benefits from the full scientific computing ecosystem without coupling this complexity to the emulator loop. Memory reads are preferred over visual processing because structured feature vectors are richer, more interpretable, and compatible with real-time interaction without GPU support.

In the full agent, the synchronous I/O loop is decoupled by three background threads. An `IOThread` polls `game_state.json` and dispatches actions every 20 ms. A `SaveWorkerThread` handles all file writes through a bounded queue of size three with a drop-oldest policy, ensuring the main thread never blocks on disk I/O. An `EventRecorderThread` accumulates structured game events (battle outcomes, map transitions, level-ups, perceptron spawns) and flushes them asynchronously to an event timeline file. This threaded design was introduced after profiling revealed synchronous checkpoint saves caused frame drops that disrupted the agent's timing.

B. State Representation and Perception Pipeline

The raw game state is represented at two levels of abstraction. The base state is a 6-dimensional vector of normalized player X and Y (divided by 255), map identifier, binary battle flag, binary menu flag, and facing direction encoded as 0–3. This is extended with 8 derived temporal features computed as frame-to-frame differences: X and Y velocity, map-change flag, battle-start and battle-end flags, menu-open and menu-close flags, and a direction-change flag. These temporal features capture dynamic transitions that the raw position vector cannot encode.

For overworld contexts, the learning state concatenates the 8 derived temporal features, the 600-dimensional normalized tile grid, and the 768-dimensional RGB palette vector, producing a **1,376-dimensional** input vector. During battle, tile data is omitted (the tilemap does not change in combat), yielding 776 dimensions. Battle-specific learning uses a compact **41-dimensional** feature vector encoding species IDs, HP ratios, level values, move IDs, PP fractions, stat stage values normalized around a neutral of 0.5, a trainer/wild flag, and normalized turn count. Party and bag contexts use 50- and 18-dimensional vectors respectively.

The atomic error signal driving learning is a weighted L2 norm of consecutive state differences, with map transitions weighted at $10\times$, battle start/end transitions at $5\times$, and direction changes at $0.5\times$ to prevent low-importance noise from dominating the signal.

C. Perceptron Architecture and Multi-Pool Pipeline

All decision-making modules are single-layer perceptrons. This architectural choice is a principled commitment to interpretability: in a single-layer perceptron, the contribution of any individual feature to any individual action can be computed and examined directly. There are no hidden layers through which the relationship between inputs and outputs becomes opaque. Each perceptron maintains a weight vector, a learnable bias $b \in [-2.0, 2.0]$, dual-timescale eligibility traces, a utility score, a familiarity counter, and an empirically discovered activation function. The forward pass computes:

$$\hat{y} = f_{\text{warped}}\left(\mathbf{w}^\top \mathbf{x} + b\right) \tag{1}$$

Perceptrons are organized into a hierarchical **multi-pool pipeline** system. Each pipeline consists of an ordered sequence of pools, where a pool is a fixed-width ($d = 8$) output layer containing a variable number of perceptrons sharing a semantic role. Table I defines the four pipelines. Each pipeline maintains

TABLE I
PIPELINE DEFINITIONS. ALL POOLS PRODUCE 8-DIMENSIONAL OUTPUT.

Pipeline	Layers (in order)	Depth
Overworld	spatial_awareness → area_classification → frontier_detection → objective_management → pathfinding → execution → outcome_obs	7
Battle	identification → threat_assessment → stay_or_bail → action_selection → execution → outcome_obs	6
Bag	inventory_awareness → item_selection → execution	3
Party	assessment → execution	2

its own set of perceptron modules with independent weight matrices, and credit assignment is performed at the pipeline level: only the weights of the active pipeline are updated based on any given decision step's outcome. This prevents cross-context interference—a poor battle outcome does not corrupt overworld navigation weights.

Each pool produces its 8-dimensional output via top-K aggregation: perceptrons are assigned to output dimensions by index modulo 8, and within each dimension only the $K = 3$ most strongly activated perceptrons contribute, weighted by utility scores and normalized. Pool *authority* is:

$$\alpha_{\text{pool}} = \min\left(1,\ \frac{|\{p : s_p > 0.1\}| \cdot \bar{s}_{\text{good}}}{5.0}\right) \tag{2}$$

where s_p is perceptron p's empirical fit score. Low-authority pools produce zero vectors, causing the system to fall through gracefully to Markov matching and hardcoded fallbacks. High-authority pools progressively dominate decision-making. Pipeline credit assignment uses a backward pass where the error signal is attenuated by 0.7^d for the pool d layers from the output. When a pool reaches capacity, cosine-similarity clustering (threshold 0.85) merges perceptrons by averaging weights and biases; low-utility units are paged to a residual file on disk and restored when a matching trigger context subsequently appears.

D. Hebbian Learning and Eligibility Traces

Connection strengths are updated via a Hebbian rule using dual-timescale eligibility traces. The fundamental principle—that synaptic weight between two neurons should increase when both are active simultaneously [11]—is instantiated as: when a (feature, action) pair co-activates and that action is subsequently associated with a positive outcome (successful navigation, battle win, effective item use), the corresponding perceptron weight increases proportionally to the product of feature activation and action selection probability. When co-activation is associated with a negative outcome, the weight decrements. The weight update is:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha(0.7\, e\, \mathbf{x}\, \tilde{e}_{\text{fast}} + 0.3\, e\, \mathbf{x}\, \tilde{e}_{\text{slow}}) \tag{3}$$

$$\tilde{e}_k \leftarrow \gamma_k\, \tilde{e}_k + 1.0, \quad \gamma_{\text{fast}}{=}0.5, \quad \gamma_{\text{slow}}{=}0.95 \tag{4}$$

The bias b is updated by the same scalar rule. The dual-trace formulation allows simultaneous reinforcement over short horizons (γ_{fast}) and longer temporal contexts (γ_{slow}), approximating multi-timescale credit assignment observed in biological neural circuits [14]. The learning rate α adapts per-perceptron: if mean recent error (50-step window) falls below 0.1, α decays toward 10^{-3}; if it exceeds 0.5, α grows toward 5×10^{-2}.

This mechanism differs from backpropagation: it requires no differentiable loss function, does not propagate error signals through computation graphs, and does not assume outcomes can be attributed to individual decisions with precision. Utility scores for action perceptrons are updated separately: under high stagnation, utility decays by 3% per step; under productive novelty, it grows by 2% per step, with floors at 0.05 (movement) and 0.15 (interaction) to prevent collapse.

E. Empirical Activation Function Discovery and Parametric Warping

A distinguishing feature of this system is that activation functions are not selected a priori. Each perceptron maintains a rolling buffer of 200 observations $(\text{raw dot product}, \text{observed error})$. Every 100 update steps, the perceptron evaluates all candidates in a shared `ActivationLibrary` against this buffer:

$$\text{score}(f_c) = \rho(|f_c(\mathbf{r})|,\ |\mathbf{e}|) + \min\big(0.2,\ 0.5\,\sigma_{|f_c|}\big) \tag{5}$$

where ρ is the Pearson correlation, $\mathbf{r}$ is the vector of raw dot products in the buffer, $\mathbf{e}$ the corresponding errors, and $\sigma_{|f_c|}$ the standard deviation of $|f_c(\mathbf{r})|$. A perceptron switches to a new candidate only when the candidate's score exceeds the current by a hysteresis threshold of 0.1, preventing oscillation. The standard library contains ten candidates: linear, tanh, ReLU, sigmoid, bounded linear, two soft-threshold variants (0.1 and 0.3), leaky ReLU, absolute value, and signed square.

After the selected candidate has been stable for two or more consecutive fitting cycles, a **parametric warping** stage activates. The warp applies the geometric transformation:

$$f_{\text{warped}}(x) = s \cdot f_c(t \cdot x + h) + o \tag{6}$$

where scale s, stretch t, shift h, and offset o are initialized to the identity transform $(1, 1, 0, 0)$ and refined by greedy coordinate descent with step size 0.05. The analytical derivative composes cleanly for the battle backpropagation system:

$$f'_{\text{warped}}(x) = s \cdot t \cdot f'_c(t \cdot x + h) \tag{7}$$

If a warped configuration improves the fit score by more than 0.15 over the base candidate and has diverged from the identity by at least 0.15 (sum of absolute deviations), it is registered as a new named function in the shared library, available to all subsequently spawned perceptrons. This mechanism is entirely empirically driven: activation shape is treated as a separately-fitted property of each module, distinct from both standard fixed-activation networks and end-to-end differentiable activation learning approaches that train activation parameters jointly with network weights via gradient descent. Empirical

evaluation showed that activation selection improved per-module performance by 10–20% over a uniform ReLU baseline.

F. Battle System and Two-Timescale Backpropagation

The battle subsystem operates as a separate decision thread that activates when the game's battle flag is set, bypassing the entire overworld decision stack. The battle pipeline additionally employs a two-timescale gradient learning scheme—inspired by two-timescale stochastic approximation [15]—operating alongside the Hebbian updates at two temporal granularities.

1) Turn-Level Loss: After each turn resolves (detected by a PP decrease), the following loss is backpropagated through the `action_selection` (layer 3) and `execution` (layer 4) pools:

$$\mathcal{L}_{\text{turn}} = \underbrace{-\left(\frac{d_{\text{dealt}}}{e_{\max}} - \frac{d_{\text{taken}}}{p_{\max}}\right)}_{\text{HP exchange}} \cdot \underbrace{(1 + (1 - r_p) \cdot 0.5)}_{\text{risk weight}} + 0.3\,\mathbb{1}_{\text{miss}} - 0.2\,\mathbb{1}_{\text{status}} + \mathcal{L}_{\text{ok}} \tag{8}$$

where r_p is the player HP ratio and $\mathcal{L}_{\text{ok}}$ is an overkill penalty applied when excess damage is dealt to an already-weakened enemy. A fast learning rate governs immediate within-battle tactical adaptation.

2) Battle-Level Loss: After each encounter ends, a strategic loss is backpropagated through the `identification` (layer 0), `threat_assessment` (layer 1), and `stay_or_bail` (layer 2) pools:

$$\mathcal{L}_{\text{battle}} = \begin{cases} 0.5\, r_{\text{cost}} + 0.3\, \frac{\min(T,20)}{20} & \text{win} \\ 1.0 + 0.1\, \frac{T}{20} & \text{loss} \\ 0.8\, r_{\text{cost}} & \text{run} \\ -0.5 & \text{catch} \end{cases} \tag{9}$$

where r_{cost} is fractional HP spent and T the turn count. A slow learning rate governs strategic consolidation across battles. The backpropagation signal is blended with Hebbian updates proportional to pipeline authority α (maximum blend 0.70), so that gradient updates only become significant as the pipeline develops useful representations.

3) Move Selection and Type Discovery: Move selection follows a priority chain: the battle pipeline's output (when `action_selection` pool authority > 0.2) $\rightarrow$ empirical per-species damage statistics (at least 2 observations) $\rightarrow$ ground-truth type chart from ROM extraction $\rightarrow$ empirically clustered type effectiveness (Track A) $\rightarrow$ average damage across all species. The Track A empirical discovery system accumulates per-move, per-species damage statistics and applies cosine-similarity clustering every 50 battles to produce type clusters and a cross-cluster effectiveness table. Track B (ground truth) is used only for evaluation, never during play.

A two-tier move exploration system encourages type discovery: in *desperate mode* (zero damage for ≥ 2 turns), untried moves receive a $1.5\times$ score boost; in *proactive mode* (player HP $> 60\%$, level advantage), untried moves against the current species receive a one-time $0.9\times$ boost.

4) Revenge Module: When the agent loses to the same opponent repeatedly, a Revenge Target is recorded with the losing location, enemy composition, and a target team average level equal to the enemy average level plus a margin that grows with each subsequent loss. The agent navigates back to the revenge location once the target level is reached, with strategy adjustments applied: failed move IDs are penalized, and type-advantaged moves receive a score boost.

G. Navigation System

Overworld navigation combines three complementary mechanisms. A* pathfinding operates on a learned connectivity graph built incrementally as the agent discovers area transitions. For multi-area journeys, A* plans at the map level via BFS through the connectivity graph, then executes tile-level A* within each map in sequence. Within each map, the agent probes adjacent tiles systematically to discover interactable objects, NPCs, hidden items, and exits; novel tiles receive high exploration priority. Text dialogue detection monitors a memory flag at `0x0202004F` and automatically issues button advances. A separate transition handler suppresses overworld commands during battle entry animations.

The **Adaptive Checkpoint Window System** maintains a sliding window of ± 2 nav targets around the agent's estimated current position in the demonstration sequence. Position is estimated via a three-signal fingerprint: badge count (weight $10\times$ per badge discrepancy), team average level (weight 0.5 per level discrepancy), and spatial proximity to annotated nav targets (tiebreaker). Window width adapts based on efficiency: if the agent is spending more than twice the human-equivalent frames per segment, the window widens to ± 4; if efficient, it narrows to ± 1. This ensures that only contextually appropriate demonstrations are ever available for matching, eliminating cross-stage contamination.

H. Human Demonstration Pipeline and Markov Matching

A synchronized teaching notebook records complete human gameplay using adaptive density: on action changes, a dense batch of 10 frames is logged at 2-frame intervals; during steady-state play, frames are logged sparsely at 12-frame intervals. Each recorded frame stores position, map, direction, battle/menu state, the action taken, and the 8 most recent actions.

The Markov matching algorithm scores each demonstration frame against the agent's current context:

$$\text{score} = 0.5\, S_{\text{imm}} + 0.3\, S_{\text{seq}} + 0.2\, S_{\text{par}} \tag{10}$$

Immediate similarity S_{imm} rewards same-map matches with distance-based position bonuses (exact: 0.35, ≤ 2 tiles: 0.25, ≤ 5 tiles: 0.10) plus direction and battle/menu state bonuses. Sequential similarity S_{seq} checks whether the agent's last 8 actions match the demonstration buffer (8-match: 1.0, 5-match: 0.6, 3-match: 0.3). Partial similarity S_{par} compares battle and menu state. Scores above 0.60 cause the demonstrated action to be taken; below this threshold the agent falls back to A* navigation or curiosity scoring. Separate Markov systems

operate for battle (action sequence 0.70, palette 0.20), bag (menu state 0.40, action sequence 0.35), and start-menu contexts (cursor state 0.45, action sequence 0.35).

I. Decision Priority and Action Selection

The full decision priority chain, executed at every timestep, is shown in Fig. 2. The chain proceeds through eight priority levels: party menu handling, text dialogue advancement, start menu navigation, bag interaction, battle execution, preparation state machine, and overworld decision-making. The overworld tier itself chains: Markov matching (score ≥ 0.60) $\rightarrow$ A* navigation to taught or revenge targets $\rightarrow$ curiosity-based perceptron scoring.

J. Curiosity and Stagnation Management

Curiosity serves as the residual exploration mechanism when Markov authority falls below threshold. The novelty scoring system assigns each candidate action a score based on how infrequently the resulting state has been visited, with spatial novelty—entering a tile or map never visited—weighted most heavily. A transition ban system temporarily prohibits repeated failed transitions. An exploration debt mechanism accumulates when the agent spends extended steps in one area without new tile discovery, creating internal pressure toward novel regions.

Stagnation is addressed at three granularities. Fine-grain: the agent tracks consecutive action repetition and applies escalating penalties (learning multiplier decays from 1.0 to 0.05; utility penalized at 12 repetitions; hard-reset at 18). Medium: a 50-action sliding window detects repeating subsequences and applies proportional penalties to all actions in the loop. Coarse: a game-state hash detects positional stagnation; after 20 identical hashes the stagnation-initiating action receives a 50% utility penalty. The bounding rectangle debt system accumulates when recent positions span a very small spatial region (area < 25 tiles, density > 2 visits/tile), triggering progressive blending from the taught reference model at three tiers: Tier 1 nudges utilities; Tier 2 interpolates weights; Tier 3 hard-blends weights and biases at a 40/60 AI-to-human ratio.

K. Knowledge Accumulation and Persistent Memory

The agent maintains persistent knowledge bases that grow monotonically across sessions via JSON serialization. The move database records every move encountered including power, type, PP, and observed effects. The spatial memory records visited tiles, exit locations, NPC positions, and the area connectivity graph. The item knowledge base tracks item effects and usage contexts. The type-effectiveness database accumulates damage multiplier observations for empirical type chart reconstruction. Background saving is handled by the dedicated `SaveWorkerThread`, preventing frame-rate drops during real-time play.

L. Evaluation Methodology

Evaluation proceeds along two parallel tracks. The first measures game progression by comparing structurally identical `checkpoint_metrics.json` files from the human teaching session and AI autonomous play. Checkpoints fire on meaningful game events: first visit to a new map, end of a trainer battle, and badge acquisition. Each entry records event type, badge count, team average level, mean party HP ratio across living party members, timesteps since the previous checkpoint, and map/position.

The second track measures decision quality: stagnation ratio (fraction of timesteps in a detected stagnation state,

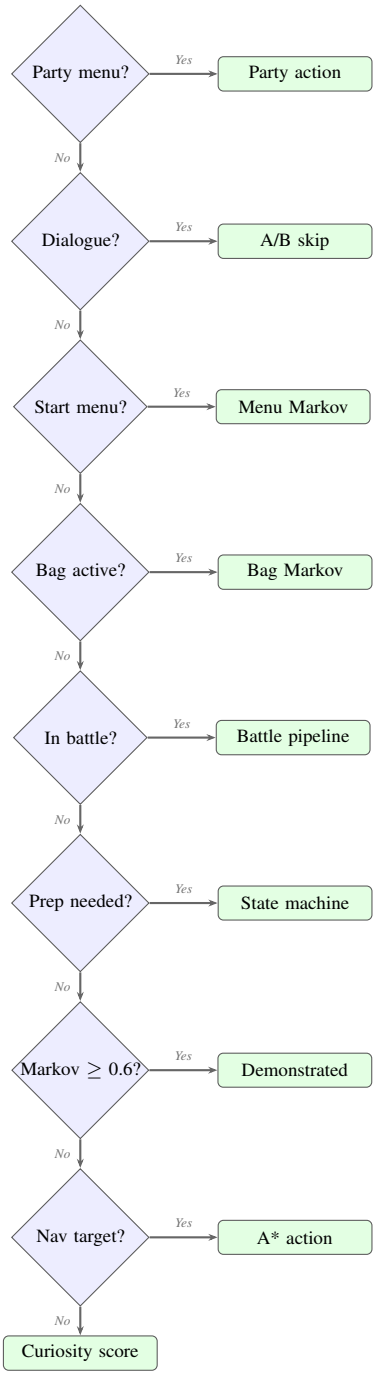

Fig. 2. Decision priority chain. The agent traverses this chain top-to-bottom at every timestep, executing the first applicable action.

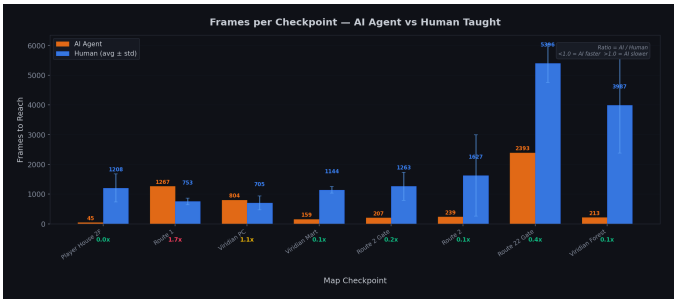

Fig. 3. Frames required to reach each map checkpoint. Orange bars show the AI agent; blue bars show the human average ± standard deviation. Ratios below the bars denote AI/Human (values below 1.0 indicate the AI arrived faster). Lower frame counts on simple transit maps reflect Markov route-following, not superior game mastery.

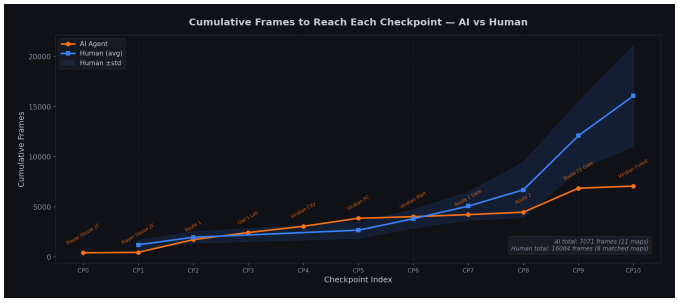

Fig. 4. Cumulative frames to each checkpoint. The AI trajectory (orange) remains consistently below the human average (blue) through the shared segment, diverging sharply at Route 22 Gate and Viridian Forest where the human allocates significant time to deliberate training and navigation.

after deduplication of overlapping concurrent events), per-map exploration coverage, battle win rate for trainer and wild encounters separately, average HP cost per battle, and move knowledge breadth (move-species pairs with ≥ 2 damage observations). Navigation efficiency is the frame count to travel between designated waypoints compared against the human demonstration baseline over more than fifty distinct waypoints spanning the game's accessible area network.

IV. Results and Evaluation

We evaluate the agent across four dimensions: game progression and navigation efficiency, party HP management, stagnation behaviour, and battle performance. All comparisons are bounded at Map 81 (Viridian Forest), the furthest location the AI reached in its 8,255-step evaluation run. The human baseline is computed from three independent teaching sessions averaging $156{,}803 \pm 28{,}800$ steps each, all of which progressed well beyond this point. Where applicable, human values are reported as mean ± standard deviation across the three runs.

A. Game Progression and Navigation Efficiency

The agent successfully traversed eleven game maps autonomously, progressing from Pallet Town (Map 13) through Route 1, Viridian City, the Viridian Pokémon Center, Viridian Mart, Route 2 Gate, Route 2, Route 22 Gate, and into Viridian Forest (Map 81), completing this segment in 7,071 total frames across the eight checkpoints shared with the human baseline. The three human demonstrators required an average of 16,084 frames to cover the same eight checkpoints.

Fig. 3 presents the per-checkpoint frame counts. On six of eight maps the AI records a lower frame count, with ratios ranging from $0.04\times$ (Player House 2F) to $1.7\times$ (Route 1). However, these raw ratios do not support the conclusion that the agent is a more efficient player — they reflect fundamentally different behavioural strategies.

1) Why the AI Appears Faster on Simple Maps: On structurally straightforward maps — Viridian Mart (AI: 159 frames vs. human: $1{,}144 \pm 110$), Route 2 Gate (207 vs. $1{,}263 \pm 474$), and Route 2 (239 vs. $1{,}627 \pm 1{,}371$) — the AI's low frame counts reflect the behaviour of the Markov matching system, not strategic efficiency. These maps contain high-density demonstration coverage of direct transit frames. The Markov matcher scores transit sequences above exploration sequences, causing the agent to select the most direct path through the area rather than engaging with NPCs, buildings, or optional encounters. The human, by contrast, deliberately allocates time in these areas for experience training, item acquisition, and environmental exploration. The AI's speed on these maps is therefore an artefact of what the demonstration corpus encodes most densely, not evidence that the agent has learned to play those maps well.

2) Where the AI Genuinely Struggles: The two maps where the gap narrows or reverses reveal the system's true limitations. On Route 1 the AI requires 1,267 frames against the human's 753 ± 107 — a $1.7\times$ slowdown reflecting battles and stagnation events on a map that requires more than simple transit. At Route 22 Gate, the AI takes 2,393 frames versus the human's $5{,}396 \pm 645$; while the ratio appears favourable ($0.4\times$), the human's higher count represents deliberate level-grinding to level 6.67 ± 0.58, whereas the AI arrives at level 10 having over-grinded on earlier maps rather than progressing strategically. Most critically, the AI's 213-frame entry into Viridian Forest records only the moment of entry: the session ends with the agent still inside the forest, unable to navigate through and exit. The human's $3{,}987 \pm 1{,}607$ frames include both entry and complete traversal. The cumulative trajectory (Fig. 4) reflects this: the AI's curve remains below the human average through the shared segment, but this compression collapses at the final two checkpoints where the human demonstrates objectives the AI has not completed.

B. Party HP Management

Fig. 5 presents average party HP ratio at the moment the agent first enters each map. The AI arrives with full HP (100%) at six of eight checkpoints. The two exceptions are Viridian PC at 64% and Route 22 Gate at 90%, both following segments where the agent engaged in wild battles.

Human demonstrators arrive at Viridian PC at 26% HP — near critical — having fought every trainer and wild encounter on Route 1. At Route 22 Gate the human average is again 26%, reflecting deliberate engagement with the high-encounter

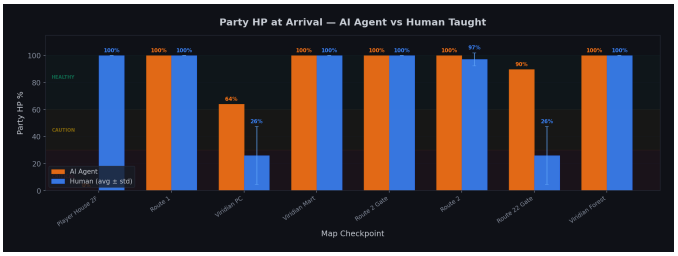

Fig. 5. Average party HP ratio at arrival at each checkpoint. The AI (orange) consistently arrives healthier than the human average (blue). This reflects reduced combat engagement rather than superior HP management: the human deliberately fights encounters that deplete HP while providing experience.

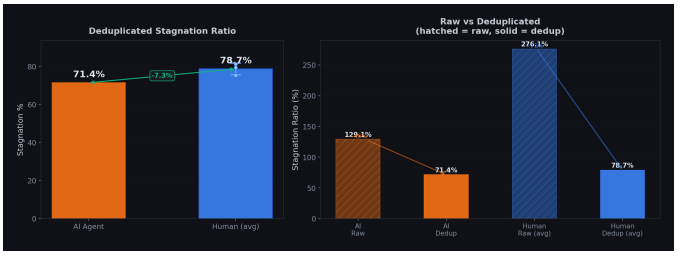

Fig. 6. Deduplicated stagnation ratio (left) and raw versus deduplicated comparison (right). The AI's 71.4% is 7.3 percentage points below the human's 78.7%, but this aggregate gap conceals qualitative differences in stagnation type that are more diagnostically relevant.

corridor for level grinding. The AI's consistently higher arrival HP is not evidence of better resource management. It reflects a fundamentally avoidance-oriented strategy: the Markov system, trained on a corpus that includes many transit frames, tends to select actions that move through encounter zones quickly rather than engaging them. The human's depleted HP accompanies higher levels at each shared checkpoint, and the AI's over-levelling through earlier encounters will not substitute for the strategic combat exposure the human accumulated by choice in high-challenge areas.

C. Stagnation Analysis

Stagnation — defined as any detected interval in which the agent is making no meaningful progress — is the primary diagnostic metric for this system. Given that the agent has no reward function and no goal-directed planner, stagnation behaviour reveals how effectively the curiosity and Markov systems recover from locally suboptimal states.

1) Overall Stagnation Ratio: After deduplicating overlapping concurrent stagnation intervals, the AI agent spent **71.4%** of its session in a stagnation state. The human baseline yielded a mean deduplicated ratio of **78.7%** ($\pm 3.1\%$). Fig. 6 presents these values alongside raw (pre-deduplication) ratios: 129.1% for the AI and 276.1% for the human average, indicating that human sessions contain far more heavily overlapping concurrent stagnation types.

The 7.3 percentage point gap in the AI's favour is marginal and should be interpreted conservatively. The human's raw overlap of 276.1% indicates that human stagnation events co-occur heavily during intentional grinding sessions. After deduplication, much of this overlap collapses. The AI's lower raw ratio reflects fewer simultaneously active stagnation types, which may indicate that the agent's stagnation events are more genuinely isolated failures rather than structured multi-type grinding behaviour.

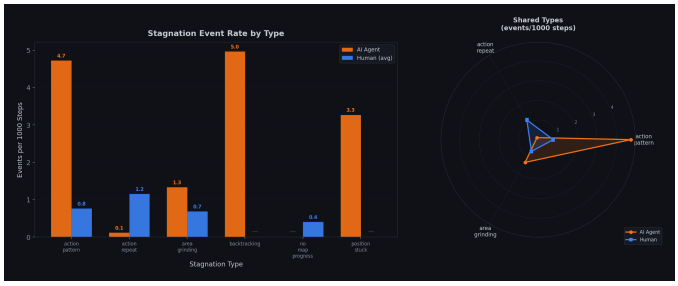

Fig. 7. Stagnation event rate by type (events per 1,000 steps). The AI (orange) exhibits two exclusive types absent from the human baseline — backtracking (5.0/1000) and position_stuck (3.3/1000) — indicating the agent becomes physically trapped or oscillates between positions, a failure mode the human never triggers.

2) Stagnation Type Breakdown: The type-level analysis (Fig. 7, normalized to events per 1,000 steps) reveals the qualitative character of AI stagnation, which is diagnostically more important than the aggregate ratio.

The AI's stagnation profile is characterized by two types that appear exclusively in the agent's data and never in the human baseline. **Backtracking** (5.0 events/1000 steps) is the AI's highest-rate stagnation type. The stagnation log confirms that events are triggered by high visit counts at specific location keys, indicating the agent is oscillating between a small set of previously visited positions. This pattern is visible in Viridian Forest, where location `(81, 40, 10)` accumulates 59 visits and `(81, 45, 20)` accumulates 36 visits in successive backtracking events. The stagnation system's anti-backtracking mechanism (`resolution: low_visit_area`) redirects the agent on individual events, but the events recur at high rate because the underlying cause — insufficient Markov coverage in structurally complex environments — persists throughout the session. **Position stuck** (3.3 events/1000 steps) indicates physical stationarity at a single tile for extended periods. The log records 27 such events with durations ranging from 20 to 260 frames. The longest event (260 frames at tile $(40, 11)$ in Viridian Forest) coincides with the terminal backtracking event, confirming that the agent's final state was one of physical entrapment near a repeated position cluster.

Among the shared stagnation types, **action pattern** events occur at 4.7/1000 steps for the AI versus 0.8/1000 for the human — a $5.9\times$ elevation consisting predominantly of single-action repetitions. **Area grinding** (1.3/1000 AI vs. 0.7/1000 human) is elevated but not dramatically so; the most significant episode is a 621-frame event in Viridian Forest during which the agent visited 161 tiles with only 2 new tiles discovered. Strikingly, **action repeat** events are dramatically lower in the AI (0.1/1000) than in the human (1.2/1000), confirming that the stagnation system's repetition penalty and utility floor mechanisms function as designed.

3) Stagnation Resolution Patterns: The stagnation log metadata records 119 total snapshots: 41 backtracking, 39 action_pattern, 27 position_stuck, 11 area_grinding, and 1 action_repeat. The dominant resolution mechanisms are `position_changed` and `low_visit_area` (for position_stuck and backtracking), and `pattern_broken` (for action_pattern events). The absence of `timeout` resolutions indicates that the stagnation system's escalating penalty mechanisms are successfully terminating most events before they reach the timeout threshold, even if they recur shortly after.

D. Battle Performance

Over the 8,255-step session the agent participated in 18 battles, winning 14, losing 1, and running from 1, yielding a win rate of **77.8%**, compared to a human baseline of **81.8%** (18W/0L/4R across the same matched segment). The 4.0 percentage point gap is notable: the human's zero losses reflect more deliberate opponent selection and type-aware move choice accumulated through longer play, while the human's higher run rate (4 vs. 1) indicates the human more readily disengages from unfavourable matchups rather than fighting to a loss. The AI's single loss occurred against a trainer encounter where move selection defaulted to a type-disadvantaged option due to insufficient empirical type-effectiveness data. The single run was executed correctly by the Markov battle system in response to a low-HP state, confirming that the flee condition threshold functions as intended.

The battle pipeline exhibits three qualitatively distinct behaviors that emerge from the move-scoring priority chain and two-timescale learning system. As per-species damage observations accumulate, the empirical type-effectiveness database shifts move selection away from the hardcoded fallback toward historically high-damage options against specific opponents. When a move has consistently produced damage near or above an opponent's maximum HP, the `action_selection` pool elevates that move's authority in future matchups against the same species, producing a one-turn knockout preference that minimizes incoming damage. The flee decision is governed by the `stay_or_bail` pool, which compares the projected damage-cost ratio against the current HP ratio; when expected incoming damage exceeds the projected kill threshold at low HP, the pool shifts authority toward the run action, producing the risk-averse HP management visible in Fig. 5.

The battle sample is small (18 encounters) and concentrated in early-game wild encounters against Lv. 2–6 opponents. The absence of trainer battles means the pipeline has not yet been evaluated against the structured opponent sequences beginning in Pewter City, and battle performance at this stage should be interpreted as confirming basic pipeline functionality rather than demonstrating strategic combat capability.

E. Summary and Limitations

Table II consolidates the primary evaluation metrics.

TABLE II
EVALUATION SUMMARY. HUMAN VALUES ARE MEAN ± STD ACROSS 3 TEACHING RUNS WHERE APPLICABLE; BATTLE STATS ARE AGGREGATED TOTALS. STAGNATION RATIO IS DEDUPLICATED.

Metric	AI Agent	Human (avg)
Total steps (eval segment)	8,255	$156{,}803 \pm 28{,}800$
Maps reached	11	> 20
Battles (W/L/Run)	14/1/1	18/0/4
Battle win rate	77.8%	81.8%
Team level at Route 22 Gate	10	6.67 ± 0.58
Stagnation ratio (dedup)	71.4%	$78.7\% \pm 3.1\%$
Backtracking (per 1000 steps)	5.0	0 (not observed)
Position stuck (per 1000 steps)	3.3	0 (not observed)
Cumulative frames (8 maps)	7,071	16,084

The evaluation reveals a system with genuine early-game capability and clear structural limitations. The Markov matching system provides effective behavioral scaffolding on maps with dense demonstration coverage, and the curiosity system extends exploration into areas not fully covered by demonstrations. The 77.8% battle win rate confirms that the battle pipeline's move-scoring and fallback chain function correctly against early-game opponents, performing comparably to the human baseline of 81.8% across the same encounter types.

The primary limitation is the agent's behaviour in structurally complex environments with sparse demonstration coverage. Viridian Forest is a multi-path map with non-trivial exit routing; once the agent exhausts its Markov matches and reverts to curiosity-driven exploration, it enters a cycle of backtracking and position_stuck events rather than discovering the exit. These failures occur in every complex map the agent enters, suggesting that the current curiosity mechanism—implemented as a symbolic novelty score over tile visit counts—is insufficient to resolve spatial navigation problems that require multi-step path planning in unexplored terrain. The agent's apparent speed advantage on simple transit maps should not be interpreted as a general capability, but rather as evidence that the Markov system successfully reproduces route-following behaviour in well-covered demonstration regions.

V. CONCLUSIONS

A. Strengths

This project demonstrates that meaningful autonomous behavior in a complex, full-length RPG is achievable without reward shaping, deep networks, or score-based optimization. The agent's full interpretability—every decision traceable to specific learned weights and discoverable activation functions—represents a genuine advantage over deep RL approaches for researchers who want to understand not just what an agent does, but why. The biologically inspired design philosophy produces a system whose components have direct analogs in cognitive science: curiosity as the driver of exploration, Hebbian associative learning as the mechanism of skill acquisition, and Markov imitation as the scaffolding that provides long-horizon behavioral structure.

The Adaptive Checkpoint Window System addresses a previously unresolved problem in long-horizon Markov imitation: cross-stage contamination. Its progress-scoped matching

approach is general enough to apply to any game with staged narrative progression. Empirical activation function discovery with parametric warping is similarly general: any system with multiple heterogeneous decision modules operating on different feature distributions could benefit from module-specific activation selection without requiring gradient-based activation training.

B. Limitations

The agent's performance is bounded by its demonstration coverage. In regions not covered by demonstrations, curiosity-driven exploration is the primary mechanism; as the early development phase showed, novelty-seeking alone cannot reliably navigate narrative-gated progression. Extending coverage to later game regions requires continued human teaching effort. The system's reliance on structured GBA memory reads means it cannot generalize to games where such memory maps are unavailable or require significant reverse-engineering. The perceptron architecture, while interpretable, has limited representational capacity relative to deep networks; highly nonlinear state-action relationships may exceed what a single linear layer with a fixed activation can represent.

C. Comparison to Standard RL

Relative to standard deep RL methods such as DQN [1], this system occupies a fundamentally different position: it requires no reward function, no differentiable objective, and no GPU-scale computation. It is not competitive with deep RL on environments where dense reward signals and computational resources are abundant. It is, however, more interpretable, more computationally accessible, and more biologically plausible. The system is best understood as an exploration of what is achievable in complex, reward-free environments using biologically-inspired principles and explicit human teaching.

D. Future Work

Several directions for future development are immediately apparent. Visual perception could be integrated via a lightweight convolutional module operating on tile sprites, adding a visual channel without the full cost of pixel-level processing. Extending team composition management to handle full six-Pokémon teams would significantly expand late-game capability. Formal ablation studies removing individual components—Markov matching, Hebbian learning, parametric warping, the Adaptive Checkpoint Window—would quantify each component's contribution more precisely than the current observational approach. Finally, the Lua–Python pipeline and multi-pool perceptron architecture are general enough to transfer to other GBA RPGs, and testing on related titles (e.g., Pokémon Emerald) would assess how much learned structure is game-specific versus transferable to the RPG genre.

Acknowledgment

The authors thank the BizHawk development community for their open-source multi-system emulator and its Lua scripting API, which made the Lua–Python interface pipeline possible. This work was completed as a final project for CS5100: Foundations of Artificial Intelligence at Northeastern University, advised by course instructors at the Khoury College of Computer Sciences.

References

[1] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[2] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, 2016.

[3] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, no. 7782, pp. 350–354, 2019.

[4] Game Freak, *Pokémon FireRed*, Nintendo Game Boy Advance, 2004.

[5] P.-Y. Oudeyer and F. Kaplan, "What is intrinsic motivation? A typology of computational approaches," *Frontiers in Neurorobotics*, vol. 1, p. 6, 2007.

[6] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 2778–2787, PMLR, 2017.

[7] M. G. Bellemare, S. Srinivas, R. Munos, and K. Kavukcuoglu, "Unifying count-based exploration and intrinsic motivation," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 29, 2016.

[8] Y. Burda, H. Edwards, A. Storkey, and O. Klimov, "Large-scale study of curiosity-driven learning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[9] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proc. Int. Conf. Artificial Intelligence and Statistics (AISTATS)*, 2011.

[10] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017.

[11] D. O. Hebb, *The Organization of Behavior: A Neuropsychological Theory*. New York: Wiley, 1949.

[12] T. Miconi, J. Clune, and K. O. Stanley, "Differentiable plasticity: Training plastic neural networks with backpropagation," in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 3559–3568, PMLR, 2018.

[13] E. Oja, "Simplified neuron model as a principal component analyzer," *Journal of Mathematical Biology*, vol. 15, no. 3, pp. 267–273, 1982.

[14] E. M. Izhikevich, "Solving the distal reward problem through linkage of STDP and dopamine signaling," *Cerebral Cortex*, vol. 17, no. 10, pp. 2443–2452, 2007.

[15] V. S. Borkar, "Stochastic approximation with two time scales," *Systems & Control Letters*, vol. 29, no. 5, pp. 291–294, 1997.